

AI Workshop 2026

ARTIFICIAL INTELLIGENCE



NSTDA 2026

Case study: Where's Waldo?

# From Pixels to Waldo

A practical tour of modern AI, YOLO object detection, and real-world fine-tuning — best practices you can actually run.



**Instructor: Teerapong Panboonyuen (P'Kao)**

<https://github.com/kaopanboonyuen/WALDO2026>



 GitHub

# Three Parts, One Big Picture

01

## The AI Landscape, 2026

From classical computer vision to deep learning, LLMs, and today's agentic, multimodal AI — and why the biggest model isn't always the right tool.

02

## Building a Waldo Detector

Dataset, EDA, training three generations of YOLO, evaluation metrics, and a live scoreboard to crown a winner.

03

## Fine-Tuning Like a Pro

Nerdy-but-runnable tuning: epochs, learning rate, custom loss weighting, class imbalance, and a competition playbook.

# How Did We Get Here? A 60-Second History of AI



Where's Waldo sits at the Deep Learning layer — small, fast, purpose-built

# Basic Computer Vision: Before the Deep Learning Era



## Edge & Corner Detection

Sobel, Canny, Harris — find where pixel intensity changes sharply.



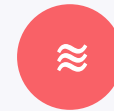
## Hand-Crafted Features

HOG, SIFT, SURF describe local texture and shape numerically.



## Template & Cascade Matching

Haar cascades slide a template across the image (classic face detectors).



## Classical Pipelines

Feature extraction → hand-tuned rules → simple classifier (SVM).

*Limitation: every feature was designed by a human. It worked for clean, narrow problems — but broke down on messy real-world images.*

# The Deep Learning Revolution (2012–2017)

## 2012

AlexNet wins ImageNet — deep CNNs beat hand-crafted features by a huge margin. The model learns its own filters.



### Convolutional Neural Networks

Layers of learned filters detect edges → textures → parts → objects.



### ResNet & Deeper Networks

Skip connections let networks go from tens to hundreds of layers.



### End-to-End Learning

One pipeline: raw pixels in, predictions out — no manual feature design.

*This is the family YOLO belongs to — a convolutional network trained end-to-end to detect objects.*

# The LLM Era: Language Becomes a Universal Interface



## Transformers & Attention (2017)

Models learn which words/tokens matter to each other — the basis of every modern LLM.



## Large Language Models

Trained on huge text corpora; can write, summarize, code, and reason step by step.



## Reasoning & Chat

Multi-turn conversation, chain-of-thought, tool use — LLMs act more like assistants.



## From Text to Everything

The same transformer recipe is now applied to images, audio, and video.

# The Vocabulary Every AI Builder Needs in 2026



## AI Agent

A single system that perceives, decides, and takes an action toward a goal (e.g. a chatbot that can book a flight).



## Agentic AI

Multiple steps or agents chained together, planning, using tools, and adapting — not just one prompt-in/answer-out call.



## Multimodal

One model that understands and generates across modalities — text, image, audio, video — in the same conversation.



## Multi-Model

A system that routes different sub-tasks to different specialized models (e.g. a small detector + a big LLM working together).

# AI Ethics & Governance: Building Responsibly



## Fairness & Bias

Does the model perform equally well across groups, lighting conditions, and data sources?



## Privacy & Data Rights

What data trained the model — and did the people in it consent?



## Transparency & Explainability

Can you explain why the model made a specific prediction?

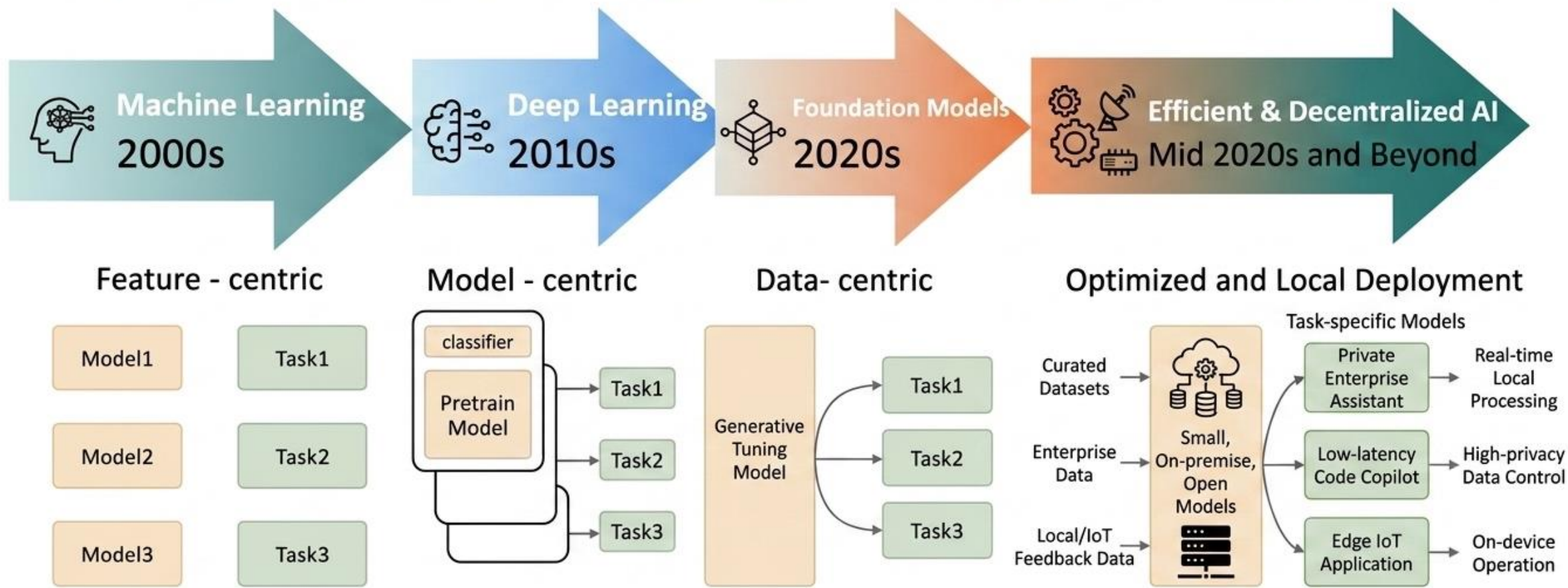


## Regulation & Governance

Policies like the EU AI Act and sector rules increasingly shape what you can deploy, and how.

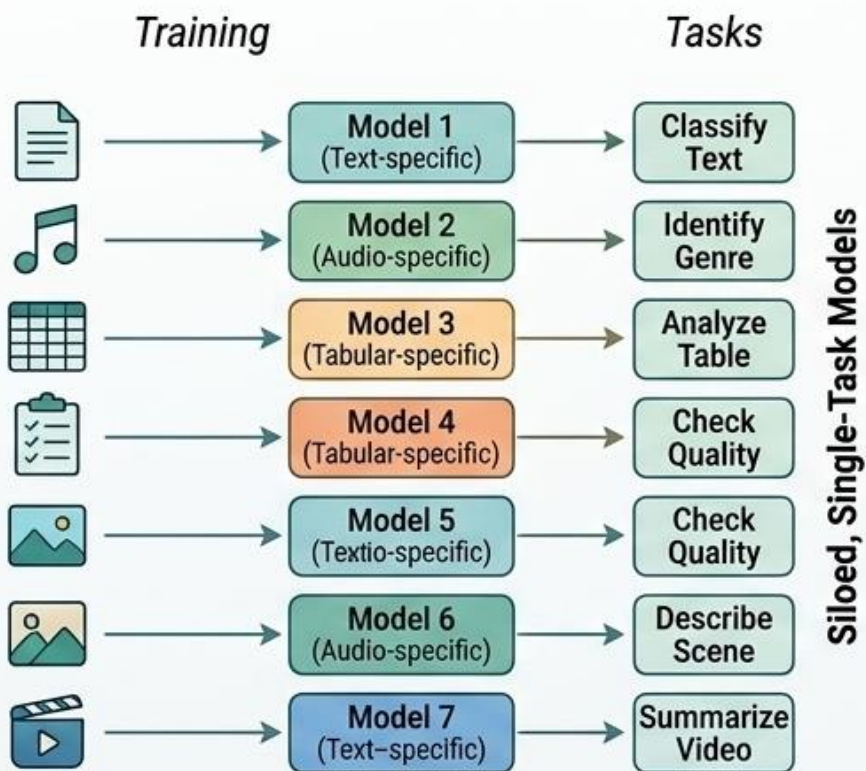
# The Next Phase of AI Evolution: Efficient and Multi-modal Models

Step function improvements and specialization over legacy AI technologies

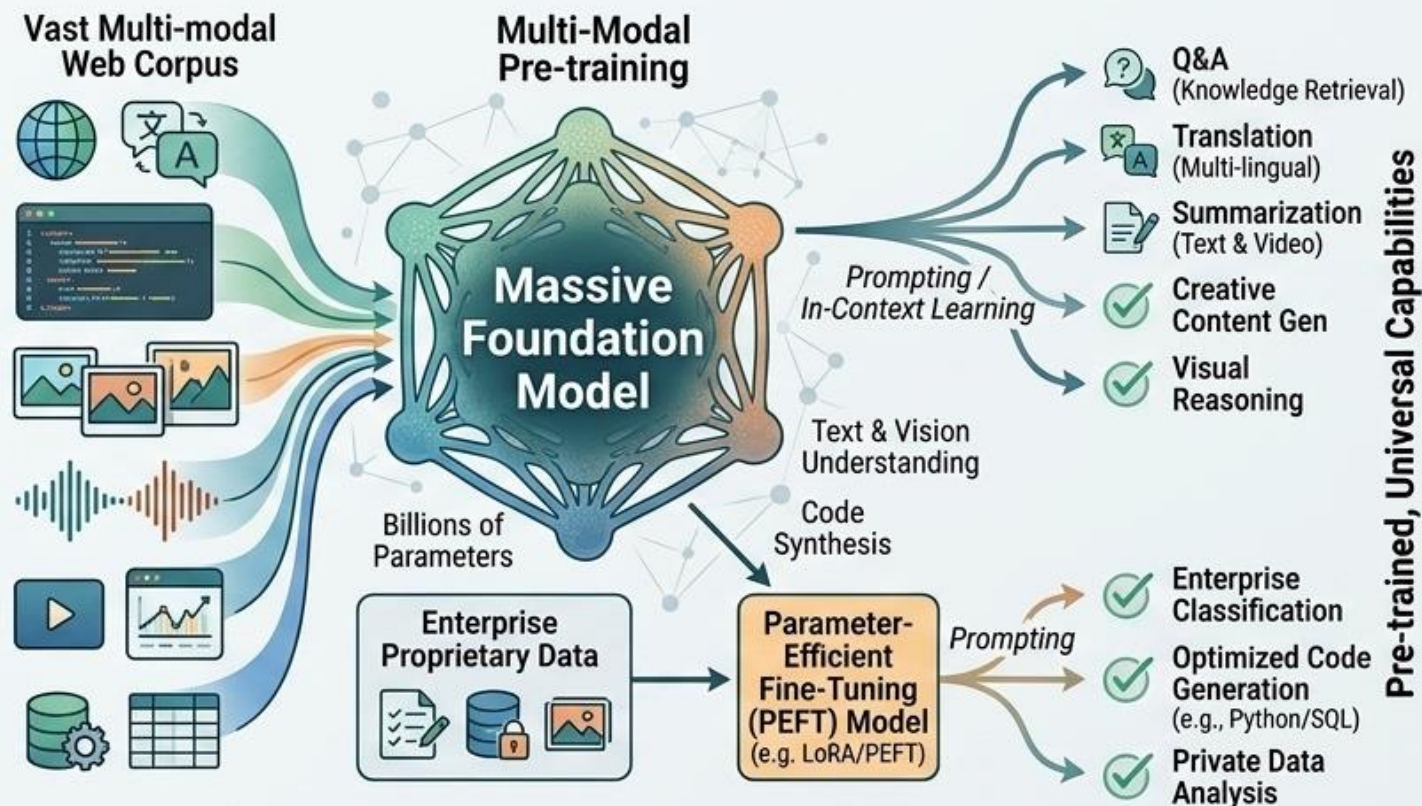


# Comparing AI Paradigms: From Task-Specific Models to Universal Foundation Models

## Traditional ML

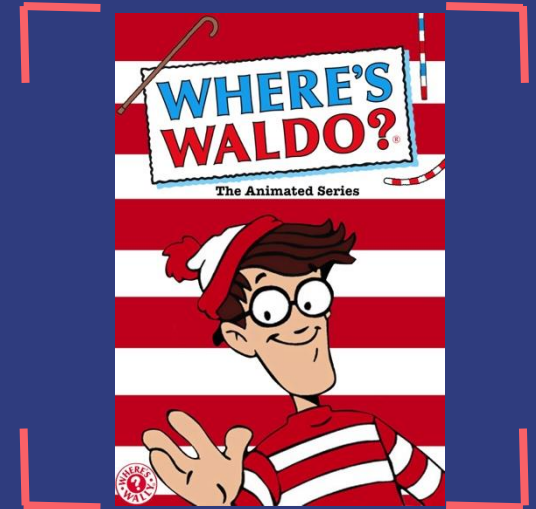


## Foundation Models



## THE BIG IDEA

# You don't need the biggest model to do a small, well-defined job.



A frontier LLM can describe a photo of a crowd in beautiful prose.

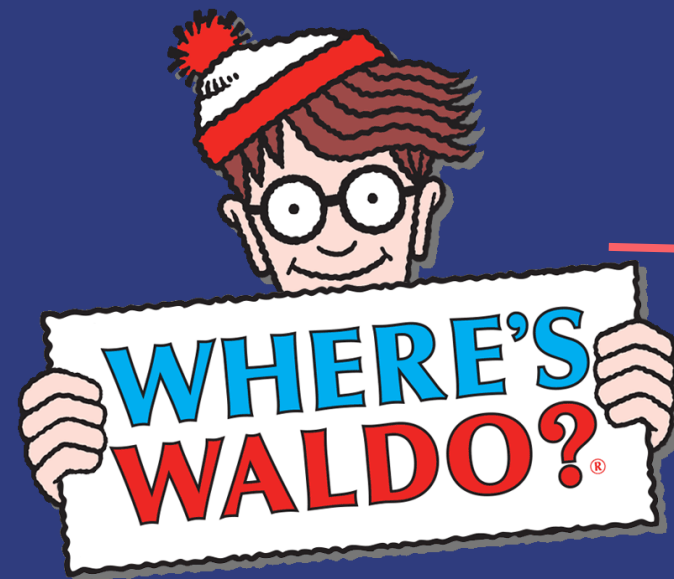
A 6-megabyte YOLO nano model can point to Waldo's exact pixel location in a few milliseconds, on a laptop CPU, for free.

Pick the smallest model that reliably solves your task — then reach for something bigger only when the task truly needs it.

## PART 2

# Building a Waldo Detector

*Where's Waldo? Teach a YOLO model to find him in milliseconds.*



# AI Tools for the Waldo Laboratory



NumPy



PyTorch



TensorFlow



pillow



NVIDIA

# What is Google Colab?



# NVIDIA Tesla T4



Google Colab  
<https://colab.research.google.com>

## GPU Architecture

Tesla T4 is a GPU card based on the acceleration.



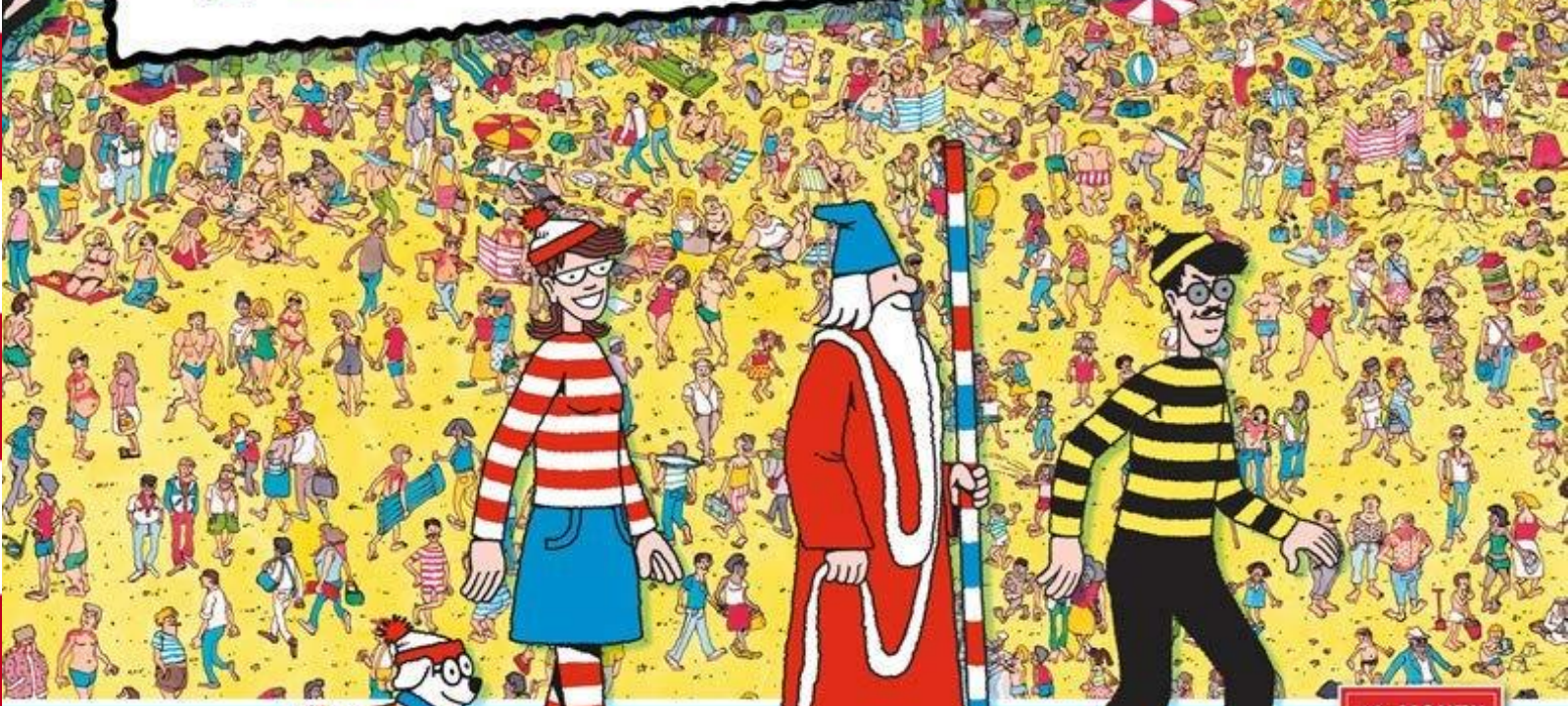
**NVIDIA**

# WHERE'S WALDO?

The Animated Series



# WHERE'S WALDO?



16-MONTH  
2021  
CALENDAR

**Waldo**, along with his trusty friend **Wenda**, travels to exciting places, always having a new adventure in every corner of the world. But wherever he goes, his sneaky rival, **Odlaw**, is never far behind, causing chaos and trying to steal the spotlight.

Luckily, Waldo can always count on the wise **Wizard Whitebeard**, who helps him out of sticky situations with his magical powers.

Together, they make an unbeatable team, solving mysteries, facing challenges, and—of course—having fun along the way!

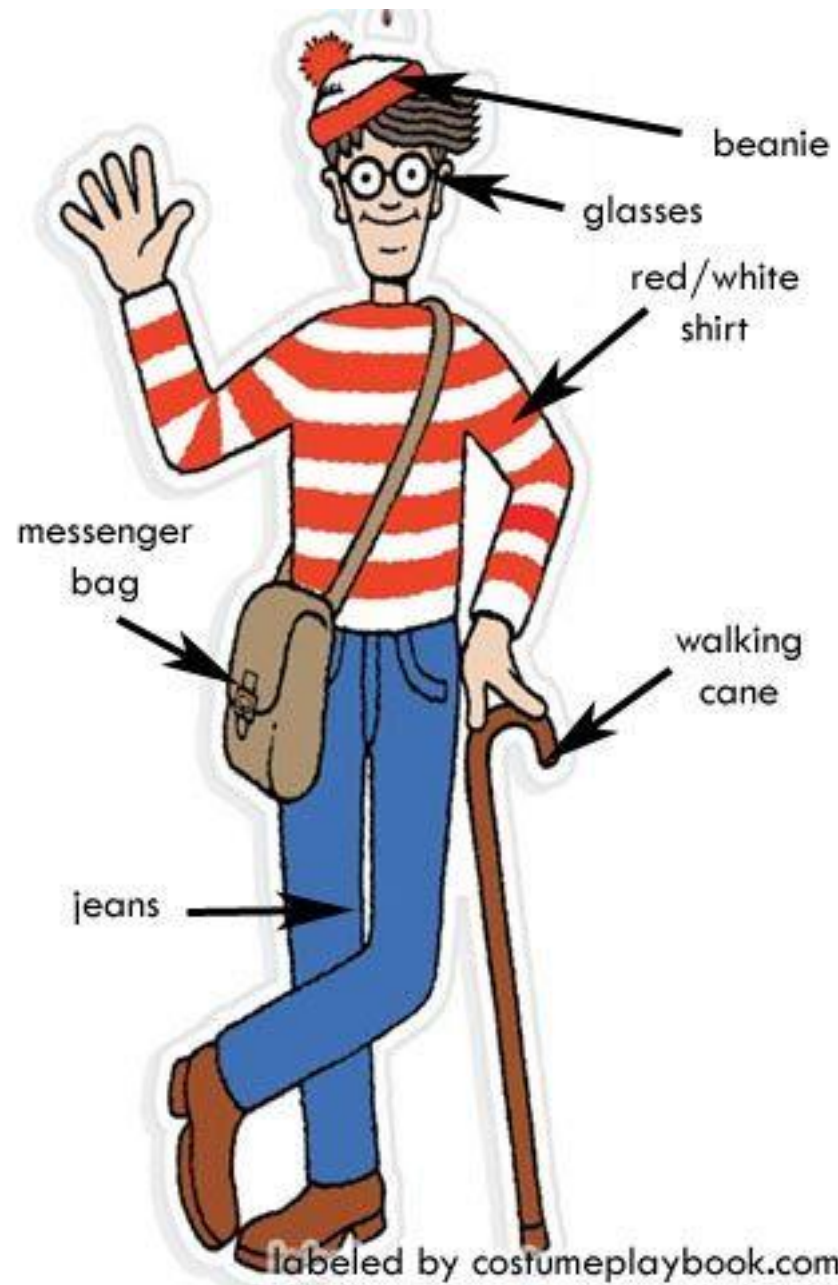
# Waldo (Wally)



**Bio:** Waldo is a brave and adventurous traveler, always sporting his red-and-white striped shirt, beanie, and round glasses. He's the kind of person who loves exploring new places—whether it's busy cities, snowy mountains, or sandy beaches—he's always on the move! But there's one thing about Waldo: he's really, really hard to find. He loves to blend into big crowds, and that's why people have been searching for him for years!

**Fun Fact:** Waldo never stops moving! He's always in search of his next big adventure, and you might just bump into him on your next journey... if you're lucky enough to spot him!

# Helpful Tips



# Wenda



**Bio:** Wenda is Waldo's best friend and fellow explorer! She shares Waldo's love of travel and has the same red-and-white striped shirt and glasses. But Wenda isn't just about looking stylish—she's clever, determined, and always ready to help out. Whether she's solving a tricky puzzle or helping Waldo escape from tricky situations, Wenda's the one you can count on to get things done.

**Fun Fact:** Wenda is always prepared for anything—she's got a bag full of gadgets and helpful tools to help her and Waldo on their adventures!

# Odlaw



**Bio:** Odlaw is Waldo's pesky rival. He wears a yellow-and-black striped outfit and is always trying to outdo Waldo, usually by causing some mischief. While Waldo is busy having fun and helping others, Odlaw is scheming to make things harder for him. He's got a mischievous mustache and loves playing pranks on Waldo and his friends!

**Fun Fact:** Odlaw's plan is always to get the spotlight away from Waldo—whether it's by hiding better than him or creating trouble in the background. But no matter what he tries, Waldo always finds a way to win!

# Wizard Whitebeard



**Bio:** Wizard Whitebeard is the magical, wise old man who always knows just what to do when Waldo and his friends need help. With his long white beard and glowing staff, Wizard Whitebeard has lived through many adventures. He uses his magic to guide Waldo and Wenda whenever they get stuck or need a little extra wisdom.

**Fun Fact:** When Wizard Whitebeard waves his staff, incredible things happen! His spells can make people float, change colors, and even make objects appear out of thin air!

Can you find Waldo in this 360 image - an illustration that went viral?

<https://blog.kuula.co/where-is-waldo-360-illustration>

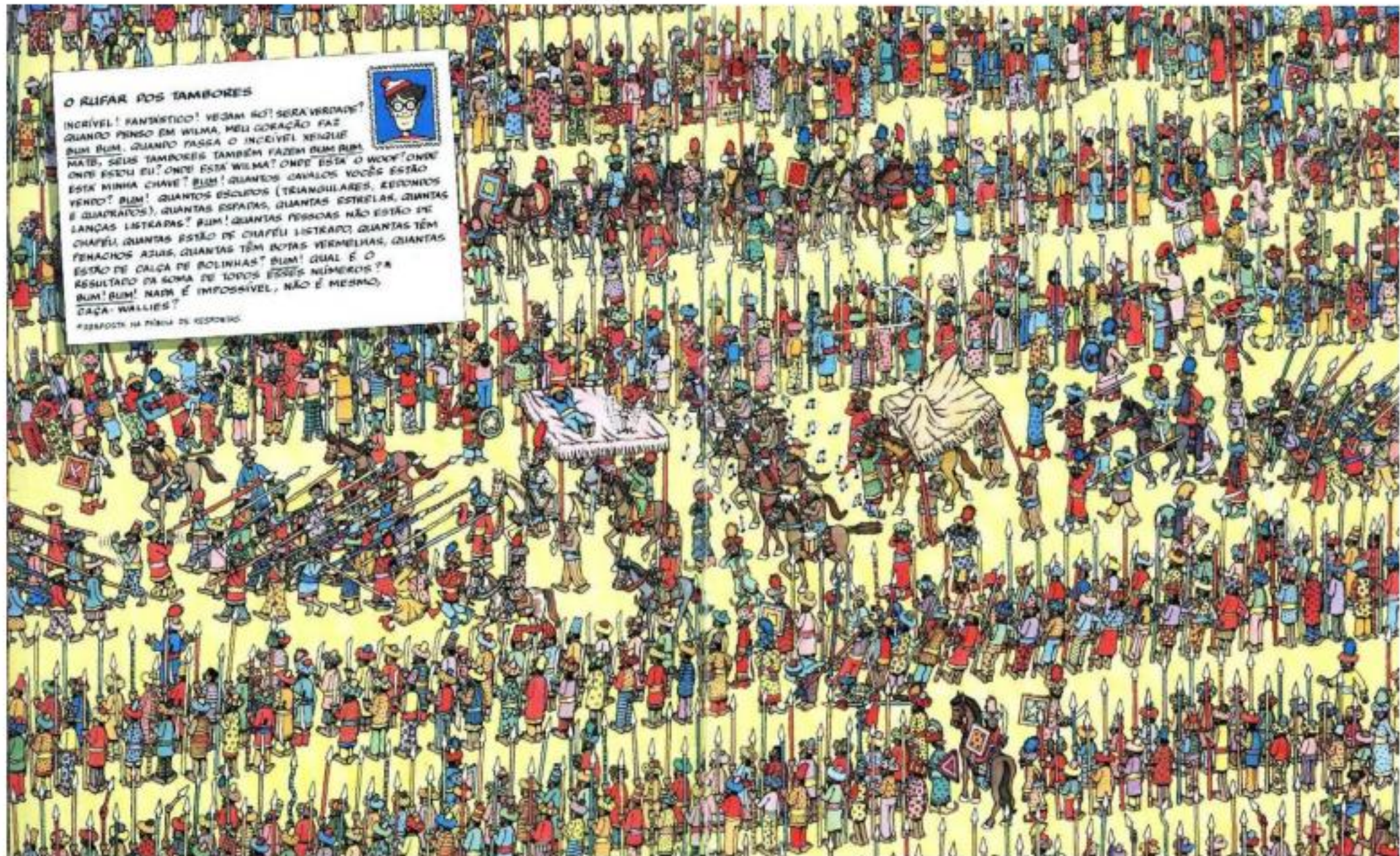


# Challenge: Fine-Tuning to Find the Winner













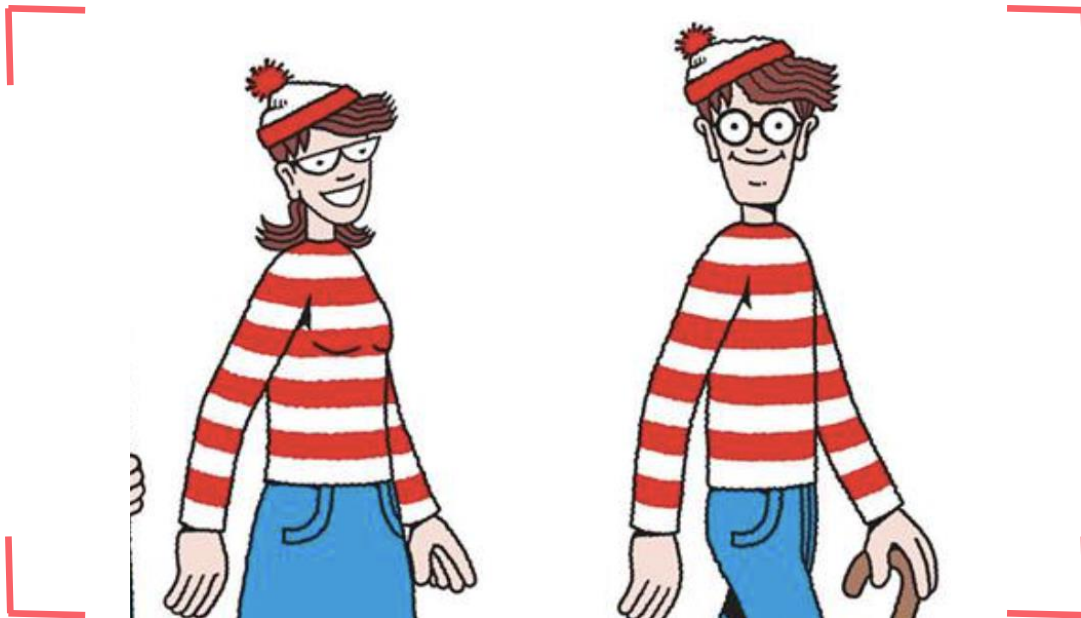


🎉🎉🎉 Total Waldo Faces Found: 1 😊  
The winner will be the student who finds the most Waldo faces! 🏆

# The Challenge: Find Waldo in a Sea of Faces

Somewhere in a crowd of thousands of tiny cartoon people, Waldo is hiding.

Instead of scanning the page for ten minutes with our own eyes, we'll teach an object detector to do it instantly.



## Today's Roadmap

- 00 Set up environment (GPU + Ultralytics)
- 01 Download & organize the dataset
- 02 EDA — understand the data first
- 03 Configure data.yaml
- 04 Train 3 YOLO architectures
- 05 Add your own 4th model
- 06 Build a scoreboard
- 07 Visualize the battle (5 views)
- 08 Crown the winner
- 09 Run it on a new test image

# What Is a Dataset, Really? The YOLO Format

A dataset = images + labels, split into train / valid folders.

```
dataset/  
  train/  
    images/ *.jpg  
    labels/ *.txt  
  valid/  
    images/  
    labels/  
data.yaml
```

One label file per image — one line per object:

```
0 0.512 0.339 0.021 0.034
```

**class\_id** 0 = waldo, 1 = wenda

**x\_center, y\_center** box center, normalized 0–1

**width, height** box size, normalized 0–1

*data.yaml simply points YOLO to these folders and lists the class names.*

# Look Before You Train: Why EDA Comes First

*Small, rare objects hiding in large images are one of the hardest cases for any detector. Seeing this early explains why architecture choice matters.*



## Count Images & Labels

How many images per split  
— enough to learn from?



## Class Balance

Is waldo rarer than  
wenda? That changes how  
we train.



## Bounding-Box Size

How big is the target  
relative to the whole  
image?



## Image Resolutions

Are source images a  
consistent size, or all over  
the place?



## Visual Sample Grid

Eyeball real images with  
ground-truth boxes  
overlaid.

# What the Data Actually Told Us

~1-2%

median box area  
vs. full image

2

object classes  
waldo · wenda

train / valid

standard YOLO  
split layout

## Takeaway

Waldo/Wenda occupy a tiny fraction of each image — this is fundamentally a small-object detection problem. That's exactly why we compare multiple YOLO generations next: newer architectures handle tiny objects and class imbalance differently.

# The Model Zoo: Three Generations, One Fair Fight

| Model    | Released | Core Idea                                          |
|----------|----------|----------------------------------------------------|
| YOLOv8n  | 2023     | The battle-tested classic — our baseline.          |
| YOLO11n  | 2024     | Ultralytics' redesigned, more efficient successor. |
| YOLOv12n | 2025     | Attention-centric architecture — the cutting edge. |

*Same data · same epochs · same image size → the comparison is fair. All use the “n” (nano) size to stay fast on a free-tier GPU.*

```
model = YOLO(cfg["weights"])
model.train(data=data_yaml, epochs=30, imgsz=640, batch=16,
            project=PROJECT_DIR, name=cfg["name"])
```

# The Training Pipeline, End to End



The “no more train2, train3” trick: every experiment gets an explicit name, and `get_latest_run_dir()` dynamically finds its folder — so you can re-run any cell as many times as you like without hand-editing paths.

# The Four Knobs You Touch First

**epochs**

30

How many full passes over the training data. Too few → underfit. Too many → overfit / wasted time.

**imgsz**

640

Input resolution fed to the model. Bigger helps tiny objects like Waldo, at the cost of speed.

**batch**

16

Images processed per training step. Larger images (bigger imgsz) usually need a smaller batch.

**device**

0 / cpu

Which GPU (or CPU) runs training. 0 = first GPU on a Colab T4.

# Evaluation Metrics That Actually Matter

## Precision

Of everything the model flagged as Waldo, how much really was Waldo?

## Recall

Of every real Waldo in the image, how many did the model actually find?

## F1

The harmonic mean of precision and recall — one number, balanced.

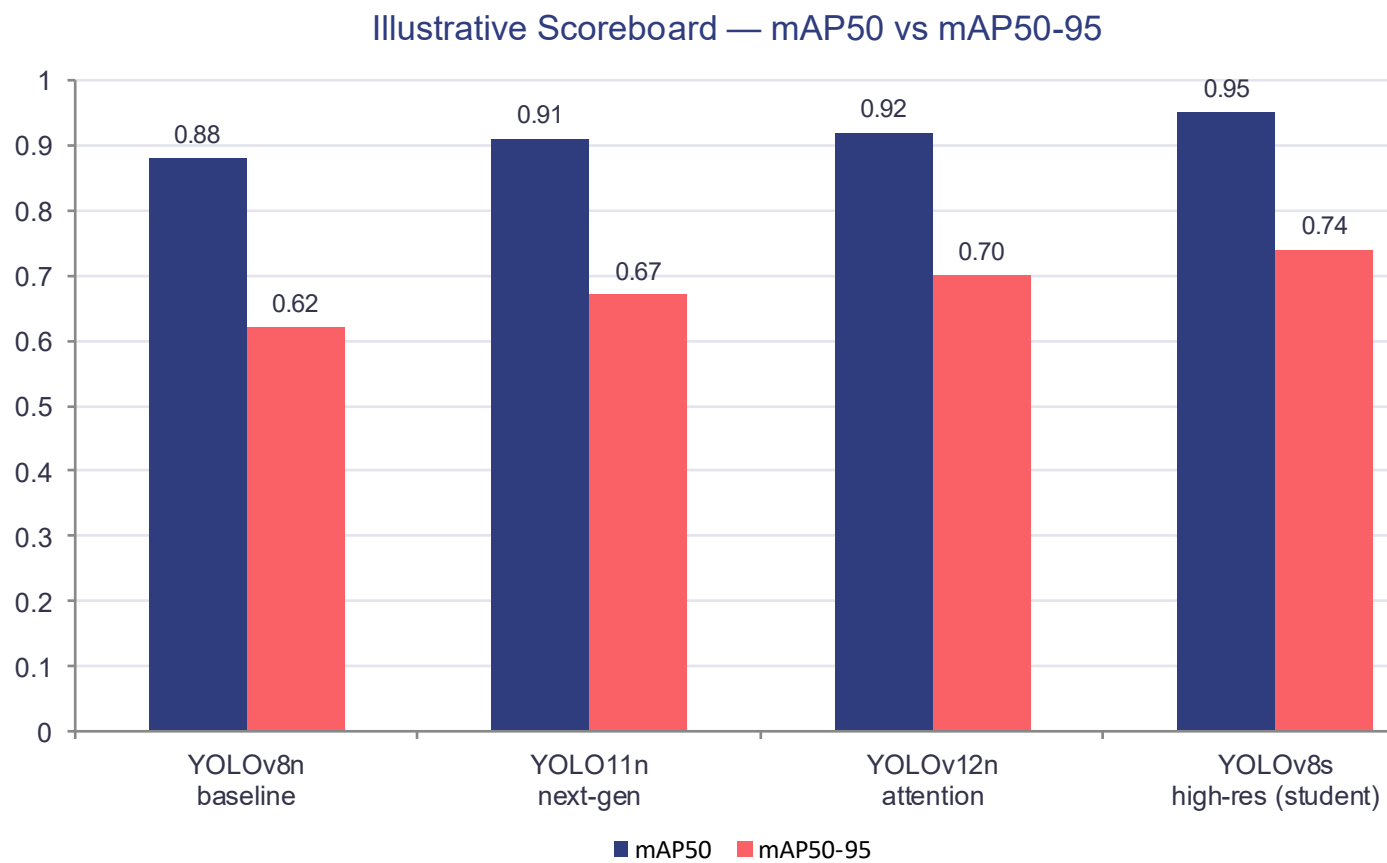
## mAP50

Mean average precision when a box only needs 50% overlap with truth to count.

## mAP50-95

Same idea, averaged over stricter overlap thresholds (50%→95%) — the gold-standard number.

# The Scoreboard: Line Every Model Up Side by Side



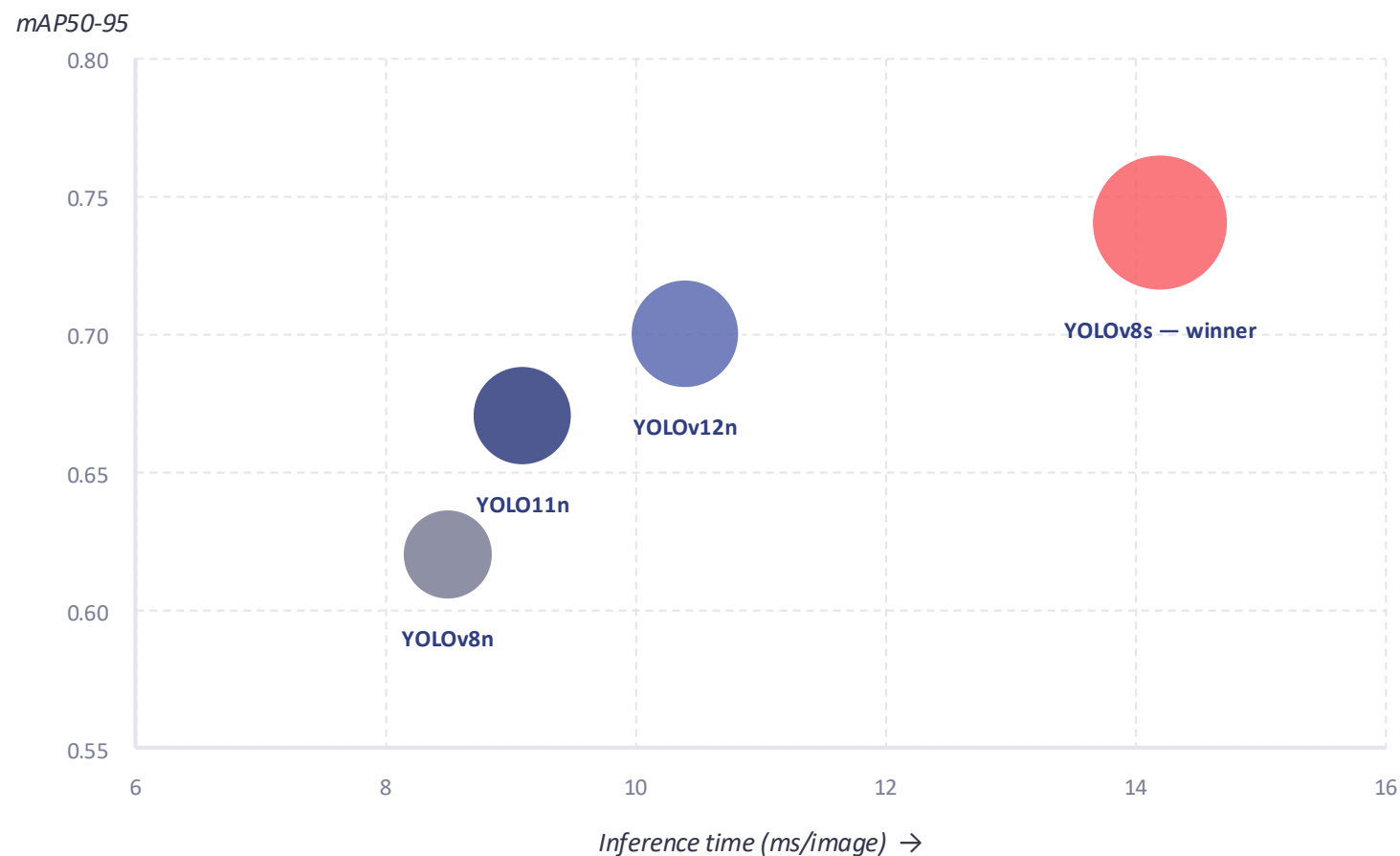
## Composite Score

$$\begin{aligned} &0.50 \cdot \text{mAP50-95} \\ &+ 0.20 \cdot \text{mAP50} \\ &+ 0.15 \cdot \text{F1} \\ &+ 0.10 \cdot \text{speed\_norm} \\ &+ 0.05 \cdot \text{size\_norm} \end{aligned}$$

*Weighted toward accuracy — but a fast, compact model still earns real credit.*

# Speed vs. Accuracy: There Is No Free Lunch

Every model trades off three things: accuracy, speed, and size. Bigger bubble = bigger file; further right = slower; higher = more accurate.



🏆 Winner

**YOLOv8s  
high-res**

Highest composite score today — bigger backbone + higher resolution paid off for tiny targets.



PART 3

# Fine-Tuning Like a Pro

*From workshop baseline to competition-grade — nerdy knobs that actually run.*



# Epochs & Learning Rate: The Two Most Sensitive Knobs

## Epochs

*One epoch = one full pass through the training set.*

1

### Too few

Model never learns the pattern (underfitting).

2

### Just right

Validation score plateaus near its best value.

3

### Too many

Training score keeps rising, validation starts falling (overfitting).

## Learning Rate (lr0)

*How big a step the optimizer takes after each batch.*

1

### Too high

Loss bounces around or diverges — never settles.

2

### Too low

Training crawls; may get stuck in a poor local minimum.

3

### Schedule it

Warm up, then decay (lrf) — fast start, stable finish.

```
patience=20    # stop early if val score
                # doesn't improve for 20 epochs
```

```
lr0=0.01, lrf=0.01
optimizer="AdamW", cos_lr=True
```

# The Nerdy-But-Runnable Parameter Cheat Sheet

| Parameter                                    | Typical Range    | What It Actually Does                                                        |
|----------------------------------------------|------------------|------------------------------------------------------------------------------|
| <code>imgsz</code>                           | 640 / 960 / 1280 | Higher helps tiny objects (like Waldo) at the cost of speed & VRAM.          |
| <code>optimizer</code>                       | SGD / AdamW      | AdamW often converges faster on small/medium datasets.                       |
| <code>mosaic</code>                          | 0.0 – 1.0        | Stitches 4 images into one — great augmentation for small-object recall.     |
| <code>close_mosaic</code>                    | 0 – 15 (epochs)  | Turns mosaic off near the end so the model sees “realistic” images last.     |
| <code>mixup</code> / <code>copy_paste</code> | 0.0 – 0.3        | Blends or pastes objects into new backgrounds — helps rare classes.          |
| <code>warmup_epochs</code>                   | 1 – 5            | Ramps the learning rate up gently so early steps don't destabilize training. |
| <code>cos_lr</code>                          | True / False     | Cosine learning-rate decay — smoother finish than a linear step-down.        |
| <code>patience</code>                        | 10 – 50 (epochs) | Early-stopping window — save GPU time once validation plateaus.              |

```
model.train(data=data.yaml, imgsz=960, optimizer="AdamW", mosaic=1.0, cos_lr=True, patience=20)
```

# Handling Class Imbalance (When Waldo Is Rarer Than Wenda)



## Check It First

Your own EDA class-balance plot tells you if this is even a problem.



## Class Weighting

Give the rare class a higher cls loss weight so mistakes on it cost more.



## Oversample the Rare Class

Duplicate or resample images containing the rare class each epoch.



## Copy-Paste Augmentation

Literally paste extra rare-class instances onto new backgrounds (copy\_paste param).



## Focal-Loss Style Weighting

Down-weight easy, well-classified examples so hard/rare ones dominate the gradient.



## Re-check the Confusion Matrix

Confirm the rare class stopped being systematically missed.

# What's Actually Inside the YOLO Loss Function?

**box**

**CloU loss**

Penalizes how far the predicted box is from the true box — position, size, and aspect ratio together.

**cls**

**BCE loss**

Binary cross-entropy for “is this class present in this box or not”.

**dfl**

**Distribution Focal Loss**

Treats each box edge as a small probability distribution for sharper localization.

```
Total loss = box_gain · box_loss + cls_gain · cls_loss + dfl_gain · dfl_loss
```

```
model.train(data=data.yaml, box=7.5, cls=0.5, dfl=1.5) # defaults — nudge cls up for imbalance
```

*Raise cls when a class is under-detected, raise box when localization (tight boxes) matters more than classification.*

# The Competition & Production Playbook



## Test-Time Augmentation

Run inference on flipped/scaled copies and average — a few % mAP for free at inference cost.



## Ensemble Checkpoints

Combine predictions from 2+ trained models; often beats any single one.



## Export for Deployment

Convert best.pt to ONNX / TensorRT / TFLite for the target device.



## Track Speed, Not Just Accuracy

A model too slow for the deployment target is not a winning model.



## Re-run EDA on Failures

Look at what the model still misses — often reveals the next data or tuning fix.



## Log Every Experiment

Explicit run names + a scoreboard — so “what changed?” always has an answer.

# Mini-Hackathon: Beat Today's Scoreboard

## Objective

Build your own YOLO experiment(s) on the same dataset and push the highest composite score you can.

## Rules

- Same data.yaml / dataset — no extra labeled data
- Any YOLO family/size: yolov8\*, yolo11\*, yolo12\*
- Tune architecture, epochs, imgsz, augmentation, optimizer, LR schedule
- No hand-labeling and no training on the test set

| Criterion                                | Weight |
|------------------------------------------|--------|
| Trains & evaluates without errors        | 30%    |
| Improvement over today's best score      | 40%    |
| Clarity of reasoning / EDA-informed      | 20%    |
| Code cleanliness, reuses dynamic helpers | 10%    |

### Ideas to try

Bigger backbone vs. more epochs on nano · higher imgsz (960/1280) for tiny targets · optimizer & LR-schedule comparisons · ensemble two checkpoints.

## KEY TAKEAWAYS

- 1 Match the model to the job — a small YOLO nano can outperform a giant LLM on a narrow, well-defined task.
- 2 EDA before training — understanding class balance and object size shapes every decision after it.
- 3 Fine-tuning is systematic — epochs, LR, loss weights, and augmentation each solve a specific failure mode.
- 4 A scoreboard beats intuition — always compare models on the same data with the same metrics.

---

**Thank you! Now go beat the scoreboard.** 